

Lesson 5 – Understanding Classes and Objects

Answers

1. Think-Share – Slide 7

A. Book – Class

Reason: Book represents a general type or design.

B. Library copy B001 – Instance

Reason: B001 is one specific copy of a Book.

C. Student – Class

Reason: Student represents a general type.

D. Learner with ID S024 – Instance

Reason: S024 represents one specific learner.

E. Car – Class

Reason: Car represents a general type.

F. Vehicle with ID C008 – Instance

Reason: C008 represents one specific vehicle.

Example sentence: “Book is a class because it represents a general type, while Library copy B001 is an instance because it is a specific object created from that type.”

2. Understanding Check #9

1. What is a class? What is an instance?

A class is a reusable design or type used to create objects. An instance is a specific object created from a class.

2. If we only define Book, how many Book instances exist?

0 Book instances.

Defining the class only creates the class. We need to call `Book()` to create an instance.

3. Can two copies of the same title be separate objects? Why?

Yes. Two copies of the same title can be separate objects because each physical copy can have its own instance, even if they came from the same `Book` class.

3. Guided Coding #16

Code:

```
class Book:  
    pass
```

```
book1 = Book()  
book2 = Book()  
book3 = Book()
```

```
print(book1)  
print(book2)  
print(book3)
```

Explanation:

- `class Book:` defines the `Book` class.
- `book1 = Book()` creates the first `Book` instance.
- `book2 = Book()` creates the second `Book` instance.
- `book3 = Book()` creates the third `Book` instance.
- The three `print()` statements display each object.

Each `Book()` call creates a separate `Book` instance.

4. Guided Coding #17

Code:

```
class Book:  
    pass
```

```
book1 = Book()
book2 = Book()
book3 = Book()

print(type(book1) is Book)
print(book1 is book2)

print(type(book2) is Book)
print(book2 is book3)
```

Expected output:

```
True
False
True
False
```

Explanation: `type(book1) is Book` is True because book1 is a Book instance. `book1 is book2` is False because they are two separate objects. The same idea applies to book2 and book3.

5. Independent Practice #19

Code:

```
class LibraryMember:
    pass

member1 = LibraryMember()
member2 = LibraryMember()

print(member1)
print(member2)

print(type(member1) is LibraryMember)
print(member1 is member2)
```

Check results:

True

False

Explanation:

member1 and member2 are both instances of the LibraryMember class, but they are separate objects. Therefore, their type check is True, while member1 is member2 is False.

6. Understanding Check #20

Given:

```
class Employee:
```

```
    pass
```

```
employee1 = Employee()
```

```
employee2 = Employee()
```

1. How many Employee classes?

1 Employee class

2. How many Employee instances?

2 Employee instances

3. What does employee1 refer to?

employee1 refers to the first Employee instance created by Employee().

4. employee1 is employee2: True or False?

False, because they are two separate instances.

Bonus:

```
employee3 = Employee()
```

This would create a third Employee instance.

7. Debugging #21

Incorrect code:

```
class book
pass

book1 = book
book2 = Book()
```

Corrected code:

```
class Book:
    pass

book1 = Book()
book2 = Book()
```

Explanation:

1. class book needs a colon :.
2. The class name should consistently be Book.
3. The class body needs indentation.
4. book1 = book only refers to the class; it does not create an instance.
5. book1 = Book() creates an actual Book instance.

The parentheses () are important because calling Book() creates an instance.

8. Debugging #22

Corrected program:

```
class Book:
    pass
```

```
book1 = Book()
```

```
book2 = Book()
```

What changed?

- Added the colon after Book.
- Indented pass.
- Used Book consistently.
- Used Book() for both book1 and book2.
- Both variables now refer to separate Book instances.

9. Exit Ticket #24

1. Explain class versus instance.

A class is a reusable design or type, while an instance is a specific object created from that class. For example, Book is a class and book1 = Book() creates an instance of that class.

2. Write a Product class and two instances.

```
class Product:
```

```
    pass
```

```
product1 = Product()
```

```
product2 = Product()
```

3. Predict product1 is product2.

False

Explanation: product1 and product2 are created using separate Product() calls, so they are two different objects.

10. Extension #26

Code:

```
class Book:
    pass

book1 = Book()
alias = book1
book2 = Book()

print(book1 is alias)
print(book1 is book2)
```

Answers:

1. How many Book instances?

2 Book instances

2. What do the checks print?

True

False

alias refers to the same object as book1, while book2 is a different object.

3. Did assignment copy anything?

No. alias = book1 gives another name to the same object; it does not create a new Book instance.

11. Extension #27

Example: School

Class: Student

Three instances:

- student1 – a student with ID S001

- student2 – a student with ID S002

- student3 – a student with ID S003

Code:

```
class Student:
    pass

student1 = Student()
student2 = Student()
student3 = Student()

print(type(student1) is Student)
print(student1 is student2)
```

Output:

True

False

Explanation: Student represents a general type, while student1, student2, and student3 are specific instances created from the class.

12. Extension #28

Original:

```
class Product:
    pass

product1 = Product
product2 = Product()

print(type(product1) is Product)
print(type(product2) is Product)
```

Predicted results:

False

True

Repair:

Change:

```
product1 = Product
```

to:

```
product1 = Product()
```

Correct code:

```
class Product:
```

```
    pass
```

```
product1 = Product()
```

```
product2 = Product()
```

```
print(type(product1) is Product)
```

```
print(type(product2) is Product)
```

Output:

True

True

Explanation: Product refers to the class itself, while Product() calls the class and creates an instance. The parentheses are important when the goal is to create an object.